

项目文件夹与代码功能详解

test1 项目文件夹与代码功能详解

本文档用于说明 test1 本地 MVP 项目的目录结构、每个文件夹的作用、每个主要代码文件实现了什么功能，以及这些代码在“恶意 APP 研判引擎”里承担什么角色。

项目根目录：

C:\Users\啤酒肚\Desktop\工作\test1

一、整体功能概览

当前 test1 实现的是一个本地可运行的恶意 APP 研判 MVP。

它的完整处理链路是：

```
360/cm Excel 数据
↓
导入 SQLite 数据库
↓
按 MD5 合并成样本
↓
四个智能体生成证据块
↓
模型甲 / 模型乙基于证据进行多轮辩论
↓
仲裁器生成 Engine C
↓
WEC 融合 Engine A、Engine B、Engine C
↓
输出最终研判结果
↓
前端页面展示结果和历史记录
```

当前模型说明：

- Engine A：360 引擎结果。
- Engine B：cm 引擎结果。
- Engine C：四智能体 + 本地 Qwen2.5-0.5B + 仲裁器生成的结果。
- 模型甲：本地 Qwen2.5-0.5B，通过“保守复核”提示词扮演。
- 模型乙：本地 Qwen2.5-0.5B，通过“风险优先”提示词扮演。
- 四个智能体：当前是规则证据智能体，不是四个独立大模型。

二、根目录文件

`run.py`

路径：

```
C:\Users\啤酒肚\Desktop\工作\test1\run.py
```

作用：

这是本地 Web 服务入口。运行它之后，会在本机启动：

```
http://127.0.0.1:8765/
```

它同时负责两类事情：

1. 提供前端页面。
2. 提供后端 API。

核心类：

```
class Handler(BaseHTTPRequestHandler)
```

它继承 Python 标准库的 `BaseHTTPRequestHandler`，不依赖 Flask/FastAPI。

主要方法：

```
do_GET()
```

处理浏览器和前端发来的 GET 请求。

已实现接口：

```
GET /api/health
```

检查服务是否正常运行。前端启动时会调用它。

```
GET /api/schema
```

返回样本字段结构说明，对应 `data/schema.json`。

```
GET /api/sample
```

返回内置示例样本，对应 `data/sample_conflict.json`。

```
GET /api/reports?limit=20
```

读取 SQLite 中保存过的研判历史。

```
GET /api/engine/stats
```

读取 360/cm 导入数据的统计信息，比如每个引擎有多少条、两个引擎重叠多少 MD5。

```
GET /api/engine/samples?limit=20&conflict_only=1
```

返回引擎样本列表。 `conflict_only=1` 表示只看两个引擎都有记录的样本。

```
GET /api/engine/sample?md5=xxx
```

根据 MD5 查询 360/cm 原始记录，并合并成系统可以研判的样本 JSON。

```
do_POST()
```

处理 POST 请求。

已实现接口：

```
POST /api/judgements
```

前端点击“开始研判”后，会把样本 JSON 发到这个接口。接口内部调用：

```
judge(payload)
```

也就是调用 `engine/pipeline.py` 里的完整研判流水线。

其他辅助方法：

```
read_json()
```

读取请求体里的 JSON。

```
serve_static()
```

返回 `web/index.html`、`web/app.js`、`web/styles.css` 等前端静态文件。

```
send_json()
```

统一返回 JSON 响应。

```
main()
```

初始化数据库，然后启动本地 HTTP 服务。

start.ps1

路径：

```
C:\Users\啤酒肚\Desktop\工作\test1\start.ps1
```

作用：

这是推荐启动脚本。

当前脚本做了几件事：

```
$env:MALAPP_USE_LOCAL_QWEN = "1"
```

启用本地 Qwen 模型。

```
$env:MALAPP_QWEN_MODEL = "Qwen/Qwen2.5-0.5B-Instruct"
```

指定使用的模型是 Qwen2.5-0.5B-Instruct 。

```
$env:MALAPP_DEBATE_ROUNDS = "2"
```

开启两轮辩论。

```
.\.venv\Scripts\python.exe -u run.py
```

使用项目虚拟环境里的 Python 启动服务。

requirements-local-qwen.txt

作用：

记录本地 Qwen 推理需要安装的 Python 依赖。

主要包括：

- torch
- transformers
- accelerate

这个文件是为了后续重装环境时可以快速恢复依赖。

README.md

作用：

项目的简要说明文件。它概括了 MVP 的目标、启动方式、API 和后续可替换模块。

注意：

当前 PowerShell 里直接查看中文时可能显示乱码，但文件本身用于说明项目，不影响系统运行。

三、engine 文件夹

路径：

```
C:\Users\啤酒肚\Desktop\工作\test1\engine
```

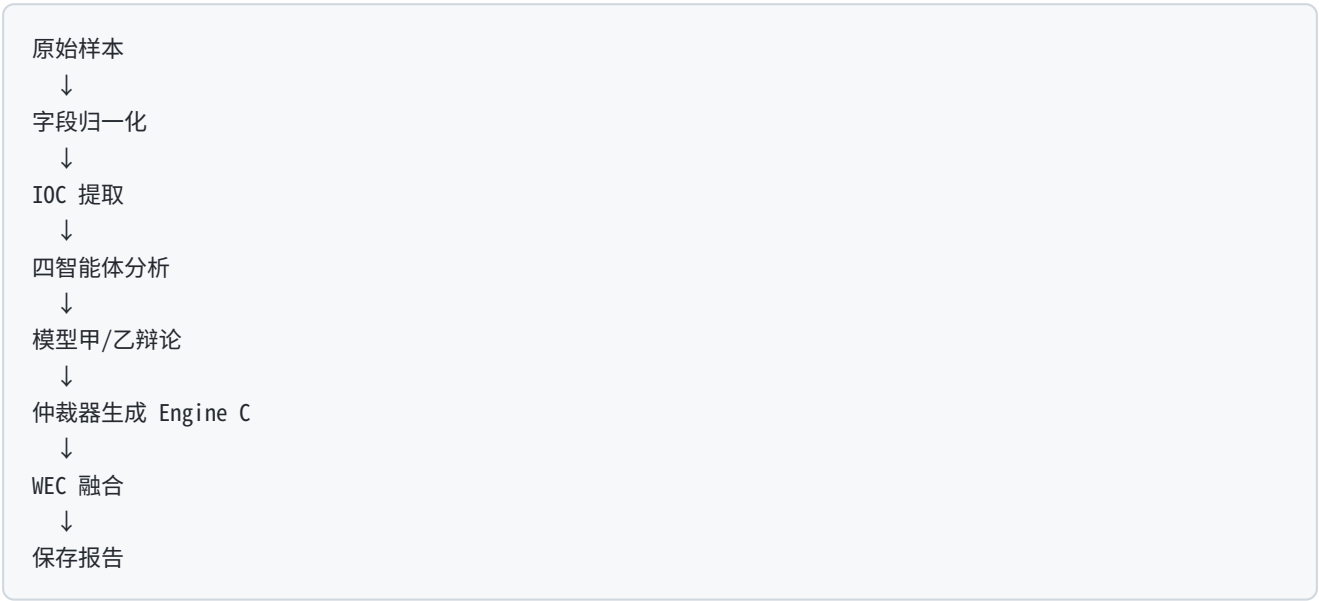
作用：

这是后端核心逻辑目录。导入数据、构建样本、四智能体、模型调用、辩论、WEC 融合、参数评估都在这里。

engine/pipeline.py

这是整个研判引擎的核心文件。

它实现的主流程是：



重要常量：

```
ROOT
DATA_DIR
DB_PATH
```

分别表示项目根目录、数据目录、SQLite 数据库路径。

```
VERDICT_LABELS
RISK_LEVEL_LABELS
```

用于把英文结论转成中文显示，例如：

```
malicious -> 恶意
suspicious -> 可疑
benign -> 良性
```

```
IOC_PATTERNS
```

定义 IOC 正则规则，用于从样本文本里提取：

- IP
- 域名
- 邮箱
- MD5
- SHA256

核心数据结构：

```
@dataclass
class EvidenceBlock
```

四个智能体都输出这个结构。

字段含义：

```
agent: 哪个智能体输出的
claim: 该智能体的判断摘要
evidence: 证据列表
confidence: 置信度
missing_fields: 缺失字段
score: 该智能体给出的风险分
```

基础工具函数：

```
utc_now()
```

生成 UTC 时间。

```
load_json()
save_json()
```

读写 JSON 文件。

```
init_db()
```

创建本地 SQLite 表 `judgements`，用于保存研判报告历史。

```
insert_report()
```

把一次完整研判报告写入 SQLite。

```
list_reports()
```

读取历史研判报告。

样本处理函数：

```
normalize_sample(raw)
```

把前端或数据导入得到的原始字段转成内部标准字段。

它会使用：

```
data/field_mapping.json
```

例如：

```
packageName -> package_name  
appName -> app_name  
engineAScore -> engine_a_score
```

如果样本没有 `sample_id`，会根据 MD5、SHA256 或原始 JSON 生成一个短 ID。

```
extract_iocs(sample)
```

把整个样本转成文本，然后用正则提取 IOC。

```
score_to_label(score)
```

把 0-100 分转成标签：

```
>=70 -> malicious  
>=45 -> suspicious  
其他 -> benign
```

四个智能体：

```
static_analysis_agent(sample)
```

静态分析智能体。

主要看：

- `signature_status`：签名是否异常。
- `permissions`：是否包含短信、通讯录、悬浮窗、安装、无障碍等高危权限。
- `packer`：是否加固、加壳、混淆。

输出：

静态特征风险分
静态证据列表
缺失字段

```
threat_intel_agent(sample, iocs)
```

情报溯源智能体。

主要看：

- `control_url`
- `download_url`
- `control_mailbox`
- `control_phone`
- IOC 数量
- `fraud_family`

如果命中 `c2`、`fraud`、`phish`、`malware` 等关键词，会提高风险分。

```
impersonation_agent(sample)
```

仿冒研判智能体。

主要看：

- `fake_app`
- `app_name`
- `package_name`
- `brand_similarity`

如果应用名或包名包含 `bank`、`wallet`、`pay`、`loan`、`finance` 等词，会认为有仿冒风险。

```
business_label_agent(sample)
```

业务打标智能体。

主要看：

- `virus_name`
- `fraud_family`

用于判断是否涉诈、钓鱼、木马、窃密等业务标签。

智能体调度：

```
run_agents(sample, iocs)
```

依次运行四个智能体，并返回四个 EvidenceBlock 。

双模型辩论：

```
debate(evidence_blocks)
```

这是模型甲、模型乙、仲裁器的核心函数。

它先根据四个智能体分数生成基础分：

- 模型甲偏保守。
- 模型乙偏风险优先。

如果启用了本地 Qwen：

```
MALAPP_USE_LOCAL_QWEN=1
```

则会调用 engine/llm_local.py 里的：

```
qwen_generate()
```

模型甲使用保守复核提示词。

模型乙使用风险优先提示词。

如果设置：

```
MALAPP_DEBATE_ROUNDS=2
```

则会进入多轮辩论：

```
run_dynamic_debate_rounds()
```

多轮辩论相关函数：

```
default_cross_examination()
```

本地模型未启用时使用的默认质询内容。

```
run_dynamic_debate_rounds()
```

执行模型甲/乙之间的动态质询和反驳。

当前两轮含义：

第 1 轮：模型乙质询模型甲，模型甲质询模型乙
第 2 轮：模型甲反驳乙的质询，模型乙反驳甲的质询

`build_challenge_prompt()`

构造“质询对方”的提示词。

`build_rebuttal_prompt()`

构造“回应质询”的提示词。

`normalize_debate_message()`

把模型输出解析成统一结构：

question
answer

`build_debate_prompt()`

构造模型甲/乙初判时使用的提示词，要求模型只输出 JSON。

WEC 融合：

`wec_decision(sample, c_score, evidence_blocks)`

实现三引擎加权融合。

输入：

- Engine A 分数：来自 360。
- Engine B 分数：来自 cm。
- Engine C 分数：来自模型甲乙辩论后的仲裁器。

它会根据证据动态调整权重：

- 情报证据强，增加 Engine C 权重。
- 仿冒证据强，增加 Engine C 权重。
- 360 和 cm 分歧大，增加 Engine C 权重。

输出：

```
final_score
verdict
risk_level
weights
engine_scores
triggered_rules
```

最终入口：

```
judge(raw_sample)
```

这是整个系统最重要的入口函数。

它被 `run.py` 的 `/api/judgements` 调用。

它完成：

1. 字段归一化。
2. IOC 提取。
3. 四智能体分析。
4. 模型辩论。
5. WEC 融合。
6. 生成完整报告。
7. 保存报告到 SQLite。
8. 返回报告给前端。

engine/llm_local.py

作用：

封装本地 Qwen2.5-0.5B 模型调用。

主要配置：

```
DEFAULT_MODEL_ID = os.getenv("MALAPP_QWEN_MODEL", "Qwen/Qwen2.5-0.5B-Instruct")
```

默认使用：

```
Qwen/Qwen2.5-0.5B-Instruct
```

并设置离线模式：

```
HF_HUB_OFFLINE=1
TRANSFORMERS_OFFLINE=1
```

这表示模型从本机缓存读取，不再联网下载。

主要函数：

```
local_qwen_enabled()
```

检查是否启用本地模型。

只有环境变量为以下值时才启用：

```
MALAPP_USE_LOCAL_QWEN=1  
MALAPP_USE_LOCAL_QWEN=true  
MALAPP_USE_LOCAL_QWEN=yes  
MALAPP_USE_LOCAL_QWEN=y
```

```
_load_model()
```

加载 tokenizer 和模型。

使用：

```
AutoTokenizer.from_pretrained()  
AutoModelForCausalLM.from_pretrained()
```

并且：

```
local_files_only=True
```

表示只从本地缓存加载。

@lru_cache(maxsize=1) 的作用是模型只加载一次，避免每次研判都重新加载。

```
qwen_generate(system_prompt, user_prompt, max_new_tokens=320)
```

真正调用 Qwen 生成文本。

当前生成参数：

```
do_sample=False  
repetition_penalty=1.05
```

含义：

- 不随机采样，输出更稳定。
- 轻微抑制重复。

```
parse_model_json(text)
```

从模型输出里提取 JSON。

它支持两种情况：

- 模型直接输出 JSON。
- 模型用 Markdown 代码块包住 JSON。

```
normalize_llm_result(raw_text, fallback_score, fallback_verdict)
```

把模型输出统一成系统需要的结构：

```
score
verdict
arguments
raw_text
```

如果模型输出格式不合格，就使用规则分数作为兜底。

engine/import_engine_data.py

作用：

把 360.xlsx 和 cm.xlsx 导入本地 SQLite 数据库。

对应数据库表：

```
engine_detections
```

为什么不用普通 openpyxl：

cm.xlsx 里很多文本是 inlineStr，普通读取方式在当前环境里不稳定，所以这个文件直接解析 .xlsx 内部 XML。

重要常量：

```
ENGINE_COLUMNS
```

把 Excel 原始列名映射成内部标准字段。

例如：

```
MD5 -> md5
pngName -> package_name
病毒名称 -> virus_name
controlUrl -> control_url
downloadUrl -> download_url
是否仿冒 -> impersonation_flag
```

```
TABLE_COLUMNS
```

定义 SQLite 表里保存哪些字段。

主要函数：

```
init_engine_tables()
```

创建 `engine_detections` 表和索引。

```
col_to_index(cell_ref)
```

把 Excel 单元格列名转成数字列号，例如 `A -> 1`，`AA -> 27`。

```
read_shared_strings()
```

读取 `.xlsx` 里的共享字符串表。

```
cell_text()
```

读取一个单元格的文本，兼容普通值和 `inlineStr`。

```
iter_xlsx_rows(path)
```

流式读取 Excel 行，不一次性把整个表加载进内存。

```
to_record(engine, headers, row)
```

把 Excel 的一行数据转换成内部字段结构。

```
import_xlsx(engine, path, limit=None)
```

导入某一个引擎的 Excel。

它使用：

```
ON CONFLICT(engine, md5) DO UPDATE
```

所以重复导入不会重复插入，会更新已有记录。

```
find_default_file(name)
```

自动在工作目录里查找 `360.xlsx` 或 `cm.xlsx`。

```
main()
```

命令行入口。

常用命令：

```
..\venv\Scripts\python.exe -m engine.import_engine_data --engine all
```

engine/engine_store.py

作用：

从 SQLite 里读取已经导入的 360/cm 数据，并合并成研判样本。

主要函数：

```
engine_stats()
```

统计导入数据：

- 每个引擎多少条。
- 两个引擎重叠多少 MD5。
- 总 MD5 数。

前端“引擎数据概览”调用的就是它。

```
get_engine_records(md5)
```

查询一个 MD5 在 360 和 cm 中的所有记录。

```
search_engine_records(limit=50, conflict_only=False)
```

列出样本列表。

`conflict_only=True` 时，只返回两个引擎都有记录的 MD5。

```
score_to_number(value)
```

把分数字段转成数字，失败时默认 50。

```
score_to_engine_label(score)
```

把 0-100 分转成：

```
malicious  
suspicious  
benign
```

```
yes_like(value)
```

判断一个字段是否表示“是”。

```
build_sample_from_engine_records(records)
```

把同一个 MD5 的 360/cm 多条记录合并成一个样本 JSON。

它做了几件事：

1. 360 作为 Engine A。
2. cm 作为 Engine B。
3. 提取应用名、包名、URL、病毒名、诈骗家族等字段。
4. 判断是否仿冒。
5. 判断是否加壳。
6. 生成 `engine_records`，保留原始引擎记录摘要。

```
build_sample_by_md5(md5)
```

按 MD5 查询数据库，然后调用 `build_sample_from_engine_records()`。

前端点击某个冲突样本时，调用的就是这个逻辑。

engine/build_dataset.py

作用：

根据导入的 360/cm 数据生成弱标注数据集。

输出目录：

```
data/datasets
```

输出文件：

```
train.jsonl  
val.jsonl  
test.jsonl
```

主要函数：

```
split_name(md5)
```

按 MD5 哈希稳定划分数据集。

比例：


```
train 约 80%
val 约 10%
test 约 10%
```

稳定划分的好处：

同一个 MD5 每次生成都会落到同一个集合里。

```
weak_label(sample)
```

生成弱标签。

注意：

这不是人工金标准，只是根据 360/cm 分数和业务字段自动推出来的标签。

大致规则：

- 有诈骗家族或仿冒分类，标成 malicious。
- 两个引擎分数都高，标成 malicious。
- 两个引擎分数都低，标成 benign。
- 两个引擎分歧很大，标成 suspicious。
- 其他中间情况标成 suspicious。

```
iter_grouped_records(conflict_only=False)
```

按 MD5 从数据库取出分组后的引擎记录。

```
build_dataset(output_dir, conflict_only=False, limit=None)
```

生成 train/val/test 三个 JSONL 文件。

每一行结构类似：

```
{
  "id": "MD5",
  "split": "train",
  "weak_label": "suspicious",
  "input": {},
  "target": {
    "verdict": "suspicious",
    "note": "weak label generated..."
  }
}
```

命令行入口：

```
main()
```

常用命令：

```
..\venv\Scripts\python.exe -m engine.build_dataset --conflict-only
```

engine/evaluate_params.py

作用：

在弱标注数据集上评估参数组合，帮助选择 WEC 和阈值参数。

它不会对 1 万多条样本逐条调用 Qwen，因为那样太慢。它使用一个快速近似的 Engine C 分数。

核心参数类：

```
@dataclass(frozen=True)
class Params
```

包含：

```
malicious_threshold: 判恶意的阈值
suspicious_threshold: 判可疑的阈值
engine_c_weight: Engine C 的权重
conflict_boost: 引擎分歧时给 Engine C 的加分
```

主要函数：

```
load_jsonl(path, limit=None)
```

读取 JSONL 数据集。

```
feature_score(sample, conflict_boost)
```

用结构化字段快速模拟 Engine C 分数。

会看：

- 360/cm 是否分歧。
- 是否仿冒。
- 是否有诈骗家族。
- 是否有 URL、邮箱、手机号等情报字段。
- 是否加壳。
- 描述里是否有风险关键词。

```
predict(sample, params)
```

用一组参数预测最终结论。

融合公式：

```
final_score = (engine_a + engine_b + engine_c * engine_c_weight) / (2 + engine_c_weight)
```

```
score_result(rows, params)
```

计算一组参数在数据集上的效果。

指标包括：

- accuracy
- malicious_precision
- malicious_recall
- risk_precision
- risk_recall
- confusion matrix

```
param_grid()
```

生成候选参数组合。

当前搜索范围：

```
malicious_threshold: 0.80 / 0.85 / 0.90  
suspicious_threshold: 0.50 / 0.55 / 0.60 / 0.65  
engine_c_weight: 0.6 / 0.8 / 1.0 / 1.2  
conflict_boost: 0.0 / 0.1 / 0.2 / 0.3
```

```
write_summary()
```

输出参数排行榜：

```
data/eval/param_grid_summary.csv
```

```
write_review_candidates()
```

输出建议人工复核样本：

```
data/eval/review_candidates.csv
```

它优先挑：

- 系统预测和弱标签不一致的样本。
- 360/cm 高分歧样本。
- 高风险样本。
- 分数在阈值边界附近的样本。

常用命令：

```
.\.venv\Scripts\python.exe -m engine.evaluate_params --dataset data\datasets\val.jsonl
```

engine/__init__.py

作用：

把 engine 目录声明成 Python 包。

这样才能使用：

```
from engine.pipeline import judge
```

或命令：

```
.\.venv\Scripts\python.exe -m engine.build_dataset
```

四、web 文件夹

路径：

```
C:\Users\啤酒肚\Desktop\工作\test1\web
```

作用：

本地浏览器页面，也就是你访问：

```
http://127.0.0.1:8765/
```

看到的页面。

web/index.html

作用：

定义页面结构。

主要区域：

顶部标题栏
样本输入区
引擎数据概览
冲突样本列表
开始研判按钮
研判结果区
A/B/C 三引擎结果
专家证据块
双模型辩论
WEC 权重
本地报告历史

关键 DOM 节点：

sampleInput: 样本 JSON 输入框
serviceStatus: 服务状态
judgeBtn: 开始研判按钮
engineStats: 引擎统计显示
engineSampleList: 冲突样本列表
verdict: 最终结论
score: 最终分数
riskLevel: 风险等级
engineResultCards: A/B/C 三引擎卡片
evidenceList: 四智能体证据块
debateList: 模型甲乙辩论展示
weightsBox: WEC 权重 JSON
historyList: 历史报告

web/app.js

作用：

前端交互逻辑。

它负责：

1. 调用后端 API。
2. 加载示例样本。
3. 加载真实引擎样本。
4. 提交研判请求。
5. 渲染研判结果。
6. 展示 A/B/C 引擎结果。
7. 展示四智能体证据。
8. 展示模型甲乙多轮辩论。
9. 展示历史报告。

主要变量：

```
const sampleInput = document.querySelector("#sampleInput");
const serviceStatus = document.querySelector("#serviceStatus");
const judgeBtn = document.querySelector("#judgeBtn");
```

这些变量绑定页面上的按钮、输入框和展示区域。

```
const t = {}
```

前端中文文案字典。很多中文用 Unicode 转义写，是为了避免 Windows/PowerShell 编码显示问题。

主要函数：

```
api(path, options = {})
```

统一封装 fetch() 请求。

```
verdictText(value, fallback)
```

把 malicious/suspicious/benign 转成中文。

```
riskText(value, fallback)
```

把 high/medium/low 转成中文。

```
backendText(value)
```

把模型后端标识转成中文，例如：

```
local_qwen2.5_0.5b -> 本地 Qwen2.5-0.5B
rule_fallback -> 规则兜底
```

```
renderReport(report)
```

渲染完整研判报告。

它会更新：

- 报告 ID
- 最终结论
- 最终分数
- 风险等级
- WEC 权重
- A/B/C 引擎卡片
- 四智能体证据

- 模型辩论过程

```
renderDebateTurns(report, debateList)
```

渲染多轮辩论。

展示内容包括：

- 第几轮
- 质询或反驳
- 谁对谁
- 问题
- 回答

```
renderEngineCards(report)
```

渲染 Engine A、Engine B、Engine C 三张卡片。

其中：

- Engine A = 360
- Engine B = cm
- Engine C = 四智能体 + Qwen + 仲裁器

```
formatScore(value)
```

格式化分数显示。

```
loadDemo()
```

调用 `/api/sample` 加载示例样本。

```
loadEngineStats()
```

调用 `/api/engine/stats` 加载 360/cm 数据统计。

```
loadConflictSamples()
```

调用 `/api/engine/samples?conflict_only=1` 加载冲突样本列表。

```
loadEngineSample(md5)
```

按 MD5 调用 `/api/engine/sample`，把真实引擎样本加载到输入框。

```
judgeCurrent()
```

点击“开始研判”后执行。

它会：

1. 读取输入框 JSON。
2. POST 到 `/api/judgements` 。
3. 接收完整报告。
4. 调用 `renderReport()` 显示。
5. 刷新历史记录。

```
loadHistory()
```

加载本地历史报告。

```
boot()
```

页面启动函数。

页面打开后自动执行：

1. 检查服务健康状态。
2. 加载示例样本。
3. 加载引擎统计。
4. 加载冲突样本列表。
5. 加载历史报告。

```
web/styles.css
```

作用：

控制页面样式。

主要设计：

- 深色顶部栏。
- 左侧样本输入。
- 右侧研判结果。
- 卡片化展示证据、辩论和引擎结果。
- 移动端自动改成单列布局。

关键样式：

```
.workspace
```

定义主页面左右两栏布局。


```
.panel
```

定义白色面板。

```
.engine-result-cards
```

定义 A/B/C 三引擎卡片布局。

```
.mini-card
```

定义证据块和辩论块样式。

```
@media (max-width: 980px)
```

小屏幕下改成单列显示，避免内容挤压。

五、data 文件夹

路径：

```
C:\Users\啤酒肚\Desktop\工作\test1\data
```

作用：

保存配置、示例样本、SQLite 数据库、生成的数据集和评估结果。

data/schema.json

作用：

定义样本字段规范。

其中：

```
"required": ["sample_id", "engine_a_score", "engine_b_score"]
```

表示最少需要这些字段。

```
"recommended": [...]
```

表示建议提供的字段，例如 MD5、SHA256、包名、权限、URL、病毒名、诈骗家族等。

data/field_mapping.json

作用：

字段别名映射。

例如：

```
pkg -> package_name
packageName -> package_name
appName -> app_name
engineAScore -> engine_a_score
fakeApp -> fake_app
```

这个文件被 `pipeline.py` 的 `normalize_sample()` 使用。

data/sample_conflict.json

作用：

内置演示样本。

前端点击“加载示例”时读取它。

这个样本故意设置了：

- Engine A 高分。
- Engine B 低分。
- 高危权限。
- 控制 URL。
- 仿冒标记。
- 品牌相似度。
- 诈骗家族。
- 加壳标记。

目的是演示冲突场景下 Engine C 和 WEC 如何参与研判。

data/mvp.db

作用：

本地 SQLite 数据库。

里面主要有两类表：

```
judgements
```

保存每次研判报告。

```
engine_detections
```

保存从 360/cm Excel 导入的检测记录。

注意：

这是本地运行产生的数据文件，不是源码。

data/datasets

作用：

保存弱标注数据集。

典型文件：

```
train.jsonl  
val.jsonl  
test.jsonl
```

用途：

- `train`：以后训练或微调时使用。
- `val`：当前用于调参数。
- `test`：最终验收时使用，不建议反复调参。

data/eval

作用：

保存参数评估输出。

典型文件：

```
param_grid_summary.csv  
best_params.json  
review_candidates.csv
```

用途：

- `param_grid_summary.csv`：参数组合排行榜。
- `best_params.json`：当前最优参数。
- `review_candidates.csv`：建议人工复核的样本。

六、 docs 文件夹

路径：

```
C:\Users\啤酒肚\Desktop\工作\test1\docs
```

作用：

保存说明文档。

已有文档：

功能实现说明.md

说明 MVP 已经实现的主要功能。

运行流程.md

说明怎么启动服务、导入数据、生成数据集、调参数。

对话记录.md

保存之前对话中解释过的内容。

多轮辩论说明.md

说明多轮辩论的开启方式、执行流程和速度影响。

项目文件夹与代码功能详解.md

就是当前这份文档。

七、tests 文件夹

路径：

C:\Users\啤酒肚\Desktop\工作\test1\tests

作用：

保存自动化测试。

tests/test_pipeline.py

作用：

测试核心研判流水线是否能正常跑通。

测试内容：

```
sample = load_json(DATA_DIR / "sample_conflict.json")
report = judge(sample)
```

它会检查：

- 最终结论必须是 malicious/suspicious/benign 之一。
- 证据块必须有 4 个，也就是四智能体都运行了。
- 最终分数必须在 0 到 1 之间。

- 辩论结果里必须有 `model_a`。
- WEC 权重里必须有 `engine_c`。

运行命令：

```
..\venv\Scripts\python.exe -m unittest discover -s tests
```

八、`.venv` 文件夹

路径：

```
C:\Users\啤酒肚\Desktop\工作\test1\.venv
```

作用：

Python 虚拟环境。

里面安装了项目运行需要的依赖，例如：

- torch
- transformers
- accelerate

这个文件夹不是业务代码，不需要手动修改。

启动服务时使用的是：

```
..\venv\Scripts\python.exe
```

九、`__pycache__` 文件夹

作用：

Python 自动生成的字节码缓存。

不是业务代码，不需要关注，也不需要手动修改。

十、当前代码模块之间的调用关系

页面启动：

```
web/index.html
↓
web/app.js
↓
GET /api/health
GET /api/sample
GET /api/engine/stats
GET /api/engine/samples
```

点击“开始研判”：

```
web/app.js judgeCurrent()
↓
POST /api/judgements
↓
run.py Handler.do_POST()
↓
engine.pipeline.judge()
↓
normalize_sample()
↓
extract_iocs()
↓
run_agents()
↓
debate()
↓
wec_decision()
↓
insert_report()
↓
返回前端 renderReport()
```

点击真实冲突样本：

```
web/app.js loadEngineSample(md5)
↓
GET /api/engine/sample?md5=xxx
↓
run.py
↓
engine.engine_store.build_sample_by_md5()
↓
get_engine_records()
↓
build_sample_from_engine_records()
↓
返回样本 JSON 到前端输入框
```

导入 Excel：

```
engine.import_engine_data.main()
↓
find_default_file()
↓
import_xlsx()
↓
iter_xlsx_rows()
↓
to_record()
↓
写入 SQLite engine_detections
```

生成数据集:

```
engine.build_dataset.main()
↓
iter_grouped_records()
↓
build_sample_from_engine_records()
↓
weak_label()
↓
split_name()
↓
写出 train/val/test jsonl
```

调参数:

```
engine.evaluate_params.main()
↓
load_jsonl()
↓
param_grid()
↓
score_result()
↓
predict()
↓
feature_score()
↓
write_summary()
↓
write_review_candidates()
```

十一、各文件负责的功能总结

run.py

本地 HTTP 服务和 API 路由。

start.ps1

启动服务，配置本地 Qwen 和辩论轮数。

requirements-local-qwen.txt

记录本地 Qwen 推理依赖。

engine/pipeline.py

核心研判流水线：四智能体、模型辩论、WEC、报告保存。

engine/llm_local.py

本地 Qwen2.5-0.5B 加载、推理、JSON 解析。

engine/import_engine_data.py

解析 360/cm Excel，并导入 SQLite。

engine/engine_store.py

查询和合并 360/cm 引擎数据。

engine/build_dataset.py

生成弱标注 train/val/test 数据集。

engine/evaluate_params.py

评估参数组合，输出调参结果和人工复核候选样本。

web/index.html

页面结构。

web/app.js

页面交互、API 调用和结果渲染。

web/styles.css

页面样式。

data/schema.json

样本字段规范。

data/field_mapping.json

字段别名到标准字段的映射。

data/sample_conflict.json

内置演示样本。

data/mvp.db

SQLite 数据库，保存导入数据和研判历史。

tests/test_pipeline.py

核心流水线自动化测试。

十二、你日常应该重点看哪些文件

如果你想理解研判逻辑，优先看：

```
engine/pipeline.py
```

如果你想理解模型甲乙怎么调用，优先看：

```
engine/llm_local.py
```

如果你想理解 360/cm 数据怎么进系统，优先看：

```
engine/import_engine_data.py  
engine/engine_store.py
```

如果你想理解 train/val/test 和调参，优先看：

```
engine/build_dataset.py  
engine/evaluate_params.py
```

如果你想改页面，优先看：

```
web/index.html  
web/app.js  
web/styles.css
```

如果你只是想运行系统，优先看：

```
start.ps1  
docs/运行流程.md
```